

Mikrovezérlők

ESP32 üzemmódjai

Dr. Hidvégi Timót
egyetemi docens

- Elmélet
 - Csoportok
 - Ébresztési lehetőségek
 - Részletek
- Linkek

Főbb tulajdonságok

Aktív mód

- ~ 100 – 260 mA; minden periféria aktív (CPU + WiFi + BT), legnagyobb fogyasztás

Modem sleep

- ~ 2 – 20 mA, CPU aktív, rádiómodulok kikapcsolva, azonnali ébredés
- Min 80 MHz-es órajel kell a WiFi stabil működése miatt

Light sleep

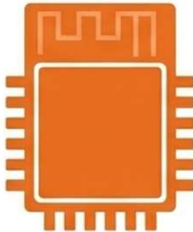
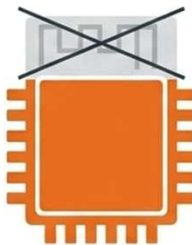
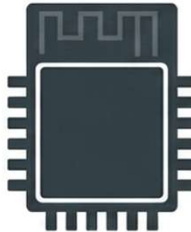
- ~ 0.8 mA, CPU megállításra került, RAM működik, gyors ébredés

Deep sleep

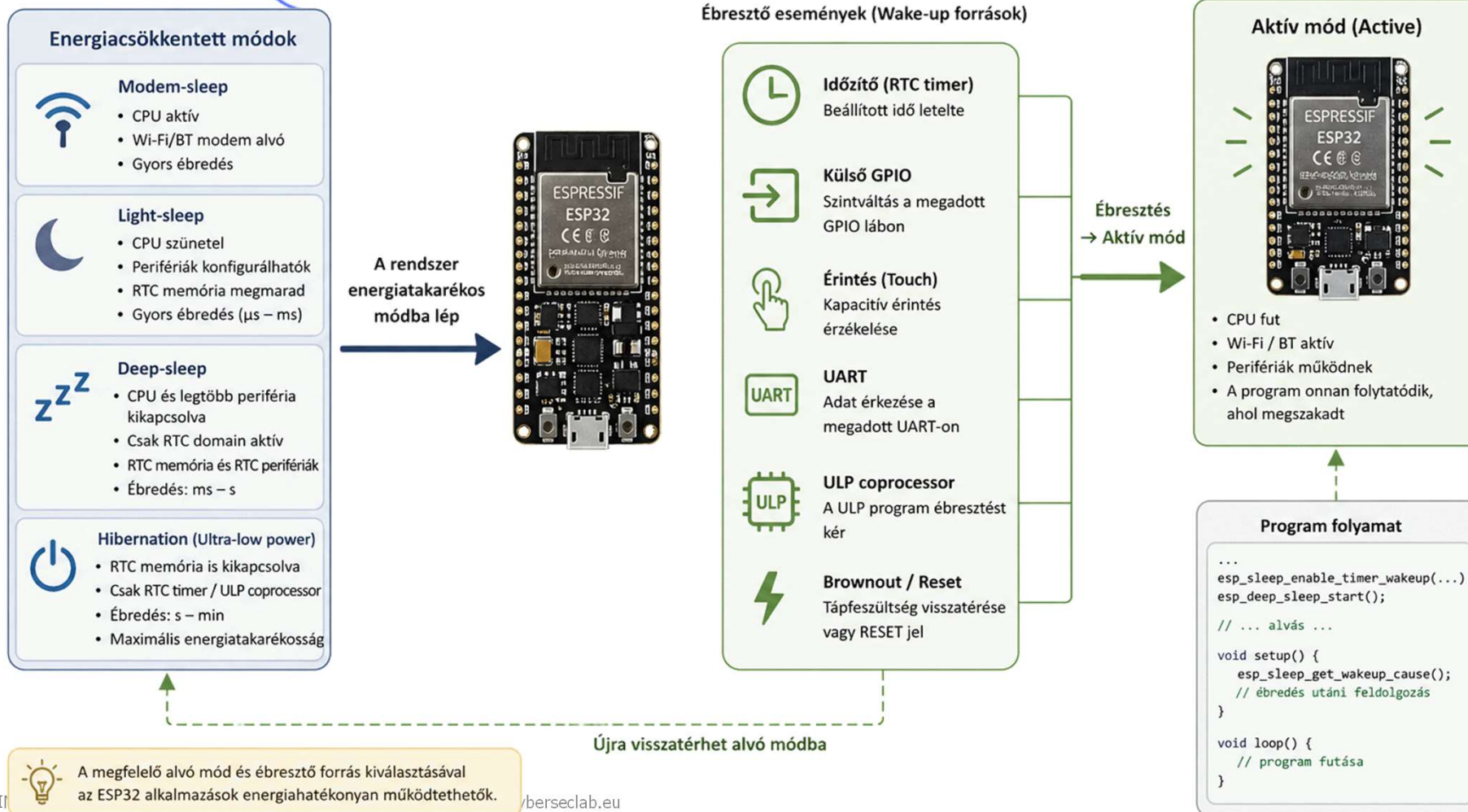
- ~ 10 – 150 uA, CPU ki, csak az RTC/ULP aktív

Hibernation

- ~ 5 uA, szinte minden kikapcsolt állapotban van (RTC időzítő működik), csak időzítő vagy külső jel ébreszt

Aktív Mód	Modem Sleep	Deep Sleep	Hibernate
Fogyasztás: 95 - 240 mA CPU: Bekapcsolva Wi-Fi/BT: Bekapcsolva	Fogyasztás: 20 - 68 mA CPU: Bekapcsolva Wi-Fi/BT: Kikapcsolva	Fogyasztás: 10 - 150 μ A CPU: Kikapcsolva RTC & ULP: Bekapcsolva	Fogyasztás: ~5 μ A Minden kikapcsolva
			

Ébresztés (wakeup) lehetőségei



DTIM (Delivery Traffic Indication Message)

- Olyan technológia és mechanizmus, amely lehetővé teszi az ESP32 számára, hogy fenntartsa a Wi-Fi kapcsolatot az AP-vel, miközben a rádiómodulját energiatakarékos célból időszakosan kikapcsolja
 - Ez a folyamat a Modem-sleep üzemmód alapja
- A DTIM működési elve
 - A DTIM lényege egyfajta „értesítési rendszer” az AP és az ESP32 között:
 - *Adatpufferelés*
 - Amikor az ESP32 Modem-sleep módban van, a router (AP) vállalja, hogy eltárolja az eszköznek szánt adatcsomagokat egy meghatározott ablak alatt, ahelyett, hogy azonnal továbbítaná őket
 - *Beacon keretek*
 - A router periodikusan úgynevezett „Beacon” kereteket sugároz. Ezek a keretek tartalmazzák a DTIM információt, amely jelzi az alvó eszközöknek, hogy van-e számukra várakozó adat a hálózaton
 - *Szinkronizált ébredés*
 - Közvetlenül a következő DTIM Beacon érkezése előtt az ESP32 rövid időre visszakapcsolja a rádióvevőjét, hogy ellenőrizze a forgalmat
 - Ha nincs várakozó adat, az RF modul azonnal visszatér alvó állapotba

DTIM (Delivery Traffic Indication Message)

■ DTIM Intervallum és ébredési ciklus

- A DTIM intervallumot az AP határozza meg, és ez adja meg azt az időtartamot, amíg az ESP32 rádiója kikapcsolva maradhat
- Ez a ciklus teszi lehetővé a Wi-Fi kapcsolat fenntartását anélkül, hogy a rádiómodul folyamatosan áramot fogyasztana

DTIM (Delivery Traffic Indication Message)

■ A Modem-sleep típusai a DTIM tükrében

- A szakirodalom két fő stratégiát különböztet meg a DTIM kezelésére, amelyek eltérő egyensúlyt kínálnak a fogyasztás és a megbízhatóság között:
- Minimum Modem-sleep
 - *Az eszköz minden egyes DTIM intervallumnál felébred, hogy fogadja a Beacont*
 - *Előnye*
 - Nem vesznek el a szórt (broadcast) adatok, mert azok közvetlenül a DTIM után érkeznek
 - *Hátránya*
 - Kevesebb energiát takarít meg, ha a DTIM intervallum rövid
- Maximum Modem-sleep
 - *Az eszköz ritkábban, egy ún. hallgatósági intervallum (listen interval) alapján ébred fel, amely több DTIM ciklust is átugorhat*
 - *Előnye*
 - Jelentősen nagyobb energiamegtakarítás
 - *Hátránya*
 - A broadcast adatok elveszhetnek, mivel az eszköz aludhat a DTIM sugárzásának pillanatában

- Ez a mód ideális olyan alkalmazásokhoz, ahol az ESP32-nek folyamatosan futtatnia kell a processzort (például egy kijelző vezérlése vagy szenzorok figyelése), de a Wi-Fi kapcsolatot is meg kell tartania anélkül, hogy folyamatosan sugározna
 - Az ESP32 fenntarthatja a hálózati kapcsolatot, miközben jelentősen csökkenti az átlagos áramfelvételt
 - Ez a mód kifejezetten olyan esetekre optimalizált, amikor az eszköz Wi-Fi állomásként (STA) csatlakozik egy hozzáférési ponthoz (AP), de nincs szüksége folyamatos adatátvitelre
 - A CPU üzemel, az órajele konfigurálható, de a rádiómodul (Wi-Fi/Bluetooth) kikapcsol
 - Hálózati kapcsolat
 - *Az eszköz képes fenntartani a kapcsolatot a hozzáférési ponttal (AP) a DTIM mechanizmus alkalmazásával, amely során a rádió periodikusan felébred az adatcsomagok ellenőrzésére*
- Beállítása
 - Az Arduino környezetben a Modem-sleep automatikusan aktiválódik a Wi-Fi csatlakozás után, ha a megfelelő alvási funkciókat meghívjuk
 - Speciálisan a WiFi.setSleep() vagy az esp_wifi_set_ps() függvénnyel konfigurálható
- Visszatérés
 - A visszatérés azonnali, nincs ébredési késleltetés
 - Az eszköz a DTIM (Delivery Traffic Indication Message) mechanizmus segítségével periodikusan felébred, hogy fogadja a router által küldött adatokat, majd ha nincs forgalom, a rádió azonnal visszaalszik

■ Energiafogyasztás és sebesség

- A fogyasztást elsősorban a CPU órajele határozza meg, mivel a rádió a legtöbb időt kikapcsolt állapotban tölti
- 240 MHz-en
 - *30–68 mA*
- 80 MHz-en (normál)
 - *20–25 mA*
- 2 MHz-en (alacsony)
 - *akár 2–4 mA-re is lecsökkenthető a fogyasztás*
- A visszatérési idő azonnali, mert a program futása nem szakad meg és nincs szükség újraindításra

■ Fontos korlátozások és feltételek

- Csak STA mód
 - *A Modem-sleep nem működik Access Point (AP) vagy sniffer módban*
- Bluetooth konfliktus
 - *A Bluetooth engedélyezése általában megakadályozza a modem-alvást, mert a Bluetooth időzítési igényei nem engedik a rádió lekapcsolását*
 - *A stabil Wi-Fi és Bluetooth funkciókhoz általában legalább 80 MHz-es CPU órajel szükséges*

- A Light-sleep során a CPU órajele leáll, és a legtöbb periféria energiatakarékos állapotba kerül
 - A RAM tartalma, a CPU állapota megmaradnak
 - *az ébredés után a program pontosan ott folytatódik, ahol megszakadt*
 - Mikor érdemes használni
 - *Olyan esetekben, ahol gyors reakcióra van szükség (pl. gombnyomás), de a köztes időben spórolni kell az energiával*
- Beállítása
 - Meg kell határozni egy ébresztési forrást (pl. időzítő vagy GPIO)
 - majd az `esp_light_sleep_start()` függvényt kell meghívni
- Visszatérés aktív módba
 - Az ébredési késleltetés kevesebb mint 1 ms
 - Ébresztési lehetőségek
 - *Időzítő (timer), bármely GPIO láb állapota, UART kommunikáció vagy érintőszensor*

- Leggyakrabban használt mód a távoli, akkumulátoros érzékelőknél
 - a fogyasztás akár 10–150 μ A-ra is lecsökkenhet
- A CPU, a fő RAM és a legtöbb periféria nem működik.
 - Csak az RTC (Real-Time Clock) vezérlő, az RTC memória és opcionálisan az ULP koprocesszor marad aktív
- Beállítása
 - Be kell állítani az ébresztési forrást, például időzítőt: `esp_sleep_enable_timer_wakeup(mikroszekundum)`
 - Meg kell hívni az `esp_deep_sleep_start()` függvényt
- Visszatérés
 - Az ébredés során az eszköz teljes újraindítást (reset) hajt végre
 - A kód a `setup()` függvény elejétől fut le újra
 - Ahhoz, hogy az adatok túléljék ezt a módot, azokat az RTC memóriában kell tárolni az `RTC_DATA_ATTR` attribútum használatával

■ Időzített ébresztés (Timer Wakeup)

- Ez a legegyszerűbb ébresztési mód, ahol az RTC vezérlő belső időzítőjét használjuk
- API függvény: `esp_sleep_enable_timer_wakeup(idő_mikroszekundumban)`
- Az időzítés pontossága a választott RTC órajeltől függ (jellemzően 150 kHz), így kisebb eltérések előfordulhatnak

■ Külső ébresztés (External Wakeup - Ext0 és Ext1)

- Két módon ébreszthető az eszköz. Csak az RTC GPIO lábak alkalmasak erre (pl. GPIO 0, 2, 4, 12-15, 25-27, 32-39 a standard ESP32-n)
- Ext0: Egyetlen láb figyelése
 - API: `esp_sleep_enable_ext0_wakeup(GPIO_NUM_X, szint)`
 - Szint: 0 = alacsony (LOW), 1 = magas (HIGH)
- Ext1: Több láb figyelése (Bitmaszk)
 - Ez a mód energiatakarékosabb (kb. 10 μ A), mert nem igényli az RTC perifériák folyamatos tápellátását
 - API: `esp_sleep_enable_ext1_wakeup(bitmaszk, mód)`
 - Módok
 - ESP_EXT1_WAKEUP_ANY_HIGH: Bármelyik kiválasztott láb magasra vált
 - ESP_EXT1_WAKEUP_ALL_LOW: Az összes kiválasztott lábnak alacsonynak kell lennie

■ Érintésalapú ébresztés (Touch Wakeup)

- Az ESP32 kapacitív érintőérzékelőit is használhatjuk ébresztésre
- API: `esp_sleep_enable_touchpad_wakeup()`
- Folyamat
 - *Először meg kell határozni egy küszöbértéket (threshold), amelynél az érzékelő jelez (érintéskor az érték csökken)*

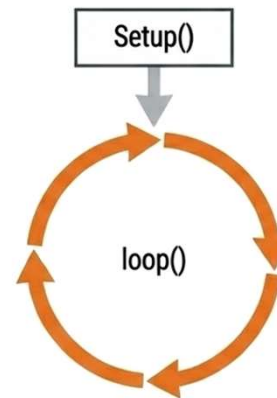
■ ULP koprocesszor

- Képes szenzorokat mérni (pl. ADC), és csak akkor ébreszteni a fő CPU-t, ha egy érték eléri a küszöböt

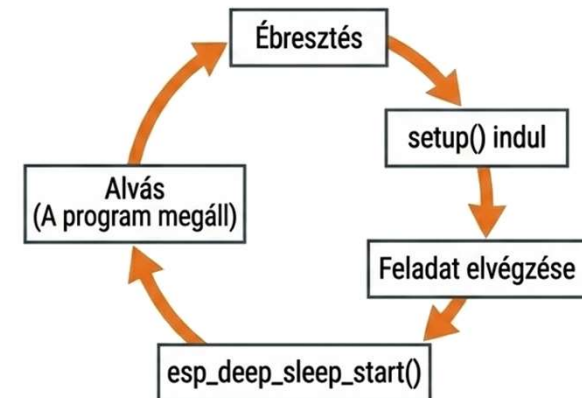
■ Adatok megőrzése: RTC Memória

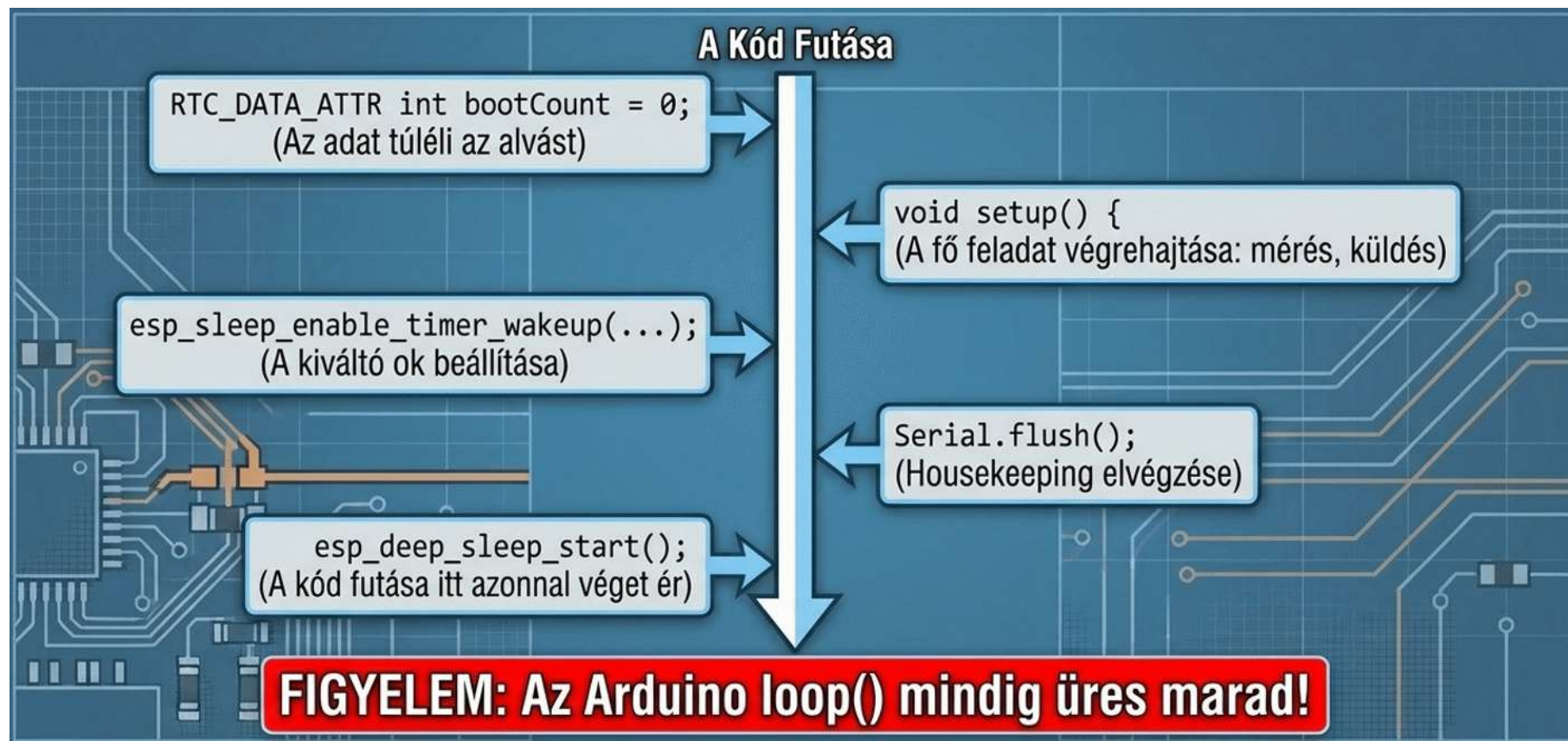
- A Deep-sleep törli a RAM-ot, az adatokat (pl. egy számlálót) az RTC memóriában kell tárolni
- Megvalósítás
 - *A változó elé az `RTC_DATA_ATTR` attribútumot kell írni*

Normál Arduino



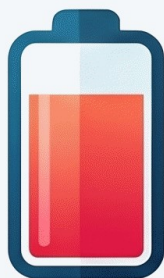
Deep Sleep Ciklus







ENERGIAFOGYASZTÁS ÉS MŰKÖDÉSI LOGIKA



Aktív Mód
160-260 mA

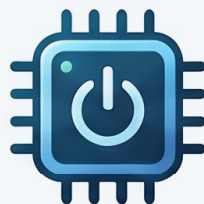


Deep Sleep
Mód

99% FELETTI

ENERGIAMEGTAKARÍTÁS

Az aktív módú 160-260 mA fogyasztás Deep Sleep módban 10-150 µA-re csökken.



A SZOFTVERES FUTÁS ALAPHÉLYZETBE ÁLL (RESET)

Ébredés után a kód a setup() elejétől indul újra; a változók értéke elveszik.

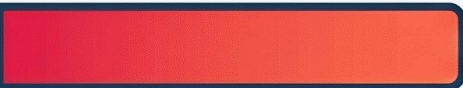


ADATMEGŐRZÉS RTC MEMÓRIÁVAL

Az RTC_DATA_ATTR jelzővel ellátott változók megőrzik értéküket a mély alvás alatt is.



**AKTÍV
(ACTIVE)**



95 - 240 mA

Aktív komponensek:
CPU, Wi-Fi, Bluetooth, Perifériák



**MODEM
ALVÁS**



20 - 68 mA

Aktív komponensek:
CPU működik, Rádió (Wi-Fi/BT) kikapcsolva



**MÉLY ALVÁS
(DEEP SLEEP)**



10 - 150 µA

Aktív komponensek:
Csak az RTC modul és az RTC memória

ÉBRESZTÉSI MÓDSZEREK (WAKE-UP SOURCES)



TIMER: IDŐZÍTETT ÉBRESZTÉS

Beállítható egy fix időintervallum (mikroszekundumban), ami után az eszköz automatikusan felébred.



TOUCH: ÉRINTÉSES ÉBRESZTÉS

Az ESP32 kapacitív érintőpinjei akkor is érzékelik a pusztá érintést, ha a CPU alszik.



EXTERNAL: KÜLSŐ TRIGGER (EXT0 / EXT1)

Külső jel (pl. nyomógomb vagy szenzor) hatására történő azonnali ébredés RTC GPIO pineken.

- A legmélyebb alvási állapot, ahol az RTC memóriát és szinte minden belső oszcillátort is lekapcsolnak
 - CPU és RAM
 - *A fő processzormagok és a teljes belső RAM áramtalanítva van*
 - Digitális perifériák
 - *Minden APB_CLK által hajtott periféria kikapcsol*
 - Belső oszcillátorok
 - *Kikapcsolják a belső 8 MHz-es oszcillátort is*
 - ULP koprocesszor
 - *Ellentétben a Deep-sleep móddal, itt az ULP koprocesszor sem működik*
 - RTC Memória
 - *Ez a legfontosabb különbség: a hibernált állapotban az RTC memória is áramtalanítva van*

■ Adatmegőrzés és visszatérés

- Adatvesztés
 - *Mivel az RTC memória nem kap tápfeszültséget, az abban tárolt változók (még az RTC_DATA_ATTR attribútummal jelöltek is) elvesznek*
- Ébredési folyamat
 - *Az eszköz ébredéskor egy úgynevezett "hideg indítást" (cold start) hajt végre*
 - *Ez egy teljes újraindulást (reset) jelent, ahol a program a legelsőtől indul, és semmilyen korábbi állapotra nem emlékszik*

■ Ébresztési források

- A visszatérés lehetőségei rendkívül korlátozottak, mivel csak az RTC vezérlő legminimálisabb része marad aktív
 - *RTC Időzítő*
 - Csak a lassú órajelen (slow clock) futó belső időzítő képes felébreszteni a chipet egy meghatározott idő után
 - *Korlátozott RTC GPIO*
 - Csak néhány specifikus láb képes ébresztést kiváltani, nem a teljes RTC GPIO készlet

- Az Arduino API-ban nincs dedikált „`hibernation_start()`” függvény
- A hibernált állapot eléréséhez a Deep-sleep indítása előtt manuálisan kell lekapcsolni a táp-domaineket az `esp_sleep_pd_config()` függvény segítségével
 - // RTC memória domain lekapcsolása
 - `esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_SLOW_MEM, ESP_PD_OPTION_OFF);`
 - `esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_FAST_MEM, ESP_PD_OPTION_OFF);`
 - // RTC periféria domain (ULP, stb.) lekapcsolása
 - `esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_OFF);`
 - // Időzített ébresztés beállítása (opcionális)
 - `esp_sleep_enable_timer_wakeup(alvási_idő_mikroszekundumban);`
 - // Alvás indítása
 - `esp_deep_sleep_start();`

- esp32 mintafeladatok elérhetősége
 - https://github.com/cyberseclabor/Mikrovezerlo_2026/tree/main/esp32

- Web
 - <https://cyberseclab.eu>
- Facebook
 - <https://www.facebook.com/IndustrialandResearchLab>
- Github
 - <https://github.com/cyberseclabor>
- LinkedIn
 - <https://www.linkedin.com/company/industrial-and-research-lab-for-cybersecurity>



SZÉCHENYI
EGYETEM
UNIVERSITY OF GYŐR
GÉPESZMÉRNÖKI, INFORMATIKAI
ÉS VILLAMOSMÉRNÖKI KAR



CYBERSECLAB

Industrial and Research Lab for Cybersecurity

enumeration ISO21434 MiTM
Artificial_Intelligence network
hacking education OT/ICS Android
car spoofing S7 forensics CyberSecLab
NIST800-82 training Purdue vehicle
HMI modell opc-ua PLC
OWASP pentest security NIS2 CAN
cyber Python C# OSINT
WiFi exploit linux AI OT nmap unit
scada sniffing kali online
modbus malware ethical
SDR Machine_Learning metasploit
vulnerability head Pentesting
Ethernet-IP